theknowledgeacademy

Elasticsearch Training

theknowledgeacademy

About Us

The world's largest provider of classroom and online training courses

- ✓ World Class Training Solutions
- ✓ Subject Matter Experts
- ✓ Highest Quality Training Material
- ✓ Accelerated Learning Techniques
- ✓ Project, Programme, and Change Management, ITIL® Consultancy
- ✓ Bespoke Tailor Made Training Solutions
- ✓ PRINCE2®, MSP®, ITIL®, Soft Skills, and More

theknowledgeacademy

Course Syllabus

Module 1: Introduction to Elasticsearch

Module 2: Install, Manage, and Secure

Module 3: Ingest Data

Module 4: Search and Analyse

Module 5: Data Management

Module 6: APIs and Developer Docs

theknowledgeacademy

Module 1:

Introduction to Elasticsearch

- ✓ What is Elasticsearch?
- ✓ Elasticsearch Basics
- ✓ Elasticsearch Use Cases

What is V-Ray? Overview and Applications

What is Elasticsearch?

🚀 **Real-Time Search Engine**

- Elasticsearch is a **distributed search and analytics engine** designed for handling large-scale, real-time data.
- Built on top of **Apache Lucene**, it enables **fast, scalable** searches across massive datasets.

⚡ **Real-Time Data Processing**

- Unlike traditional databases, Elasticsearch is optimized for **real-time data ingestion** and retrieval, making it ideal for handling dynamic, ever-growing datasets.

🔍 **Key Features**

- **Full-text search**: Ability to search unstructured text data with high speed and relevance.
- **Distributed architecture**: Easy scaling and fault tolerance with built-in clustering.
- **Multi-Tenant**: Handles data from multiple sources simultaneously with isolation.

What is V-Ray? Overview and Applications

🌍 **Widespread Adoption**

- Elasticsearch powers search and analytics for many well-known organizations, including **Netflix**, **Walmart**, **eBay**, and **GitHub**, showcasing its reliability at scale.

📈 **Scalable and Flexible**

- Elasticsearch's distributed nature allows it to easily handle large-scale deployments. It can scale horizontally by adding more nodes to the cluster without significant performance degradation.

🔐 **Open-Source**

- Elasticsearch is **open-source**, with the core functionality available for free. It also supports a vast ecosystem of plugins and extensions, making it adaptable to specific needs.

🔄 **Extensible Ecosystem**

- Beyond basic search, Elasticsearch integrates with various tools in the Elastic Stack: **Kibana** (visualization), **Logstash** (data processing), and **Beats** (data shippers).

theknowledgeacademy

Elasticsearch Basics – Data Structure

📝 **Document-Oriented Storage**

- Data is stored as **documents**, which are JSON objects containing fields and values. This allows flexibility in terms of the data structure.
  - Example document for a search index: ----->

```
{
  "title": "Elasticsearch Basics",
  "author": "Jane Doe",
  "content": "Learn the fundamentals of Elasticsearch."
}
```

📂 **Fields and Data Types**

- Documents are composed of **fields**. Each field contains **values** of specific data types: strings, integers, dates, and even complex types like arrays or nested objects.

💡 **Example of Indexing:**

- When documents are indexed, Elasticsearch maps each field to an appropriate data type to optimize search performance.

Elasticsearch Basics – Indices

📦 **What is an Index?**

- An **index** is a logical namespace that holds a collection of documents. Think of it as a **database** in traditional relational database systems, but optimized for **search** rather than transaction processing.

🗂️ **Index Mapping**

- Each index has a **mapping**, which defines the structure of the data (e.g., field types, analyzers). The mapping helps Elasticsearch determine how to store and index data for fast retrieval.

🔍 **Indexing Process**

- **Indexing** involves converting documents into a structure optimized for fast search. Documents are parsed and analyzed using **analyzers** that break down text fields into searchable terms.

🔐 **Schema-Free Design**

- While Elasticsearch supports **dynamic mapping**, which automatically guesses field types, it's often beneficial to define explicit mappings for better control over indexing and search performance.

Elasticsearch Basics – Nodes and Clusters

⚙️ **What is a Node?**

- A **node** is a single Elasticsearch instance running on a server. It can store data, execute search queries, and manage clusters.
- Nodes can be **master nodes** (managing cluster settings) or **data nodes** (holding actual data and performing search operations).

🏢 **What is a Cluster?**

- A **cluster** is a group of **nodes** that work together to provide high availability, fault tolerance, and scalability.
- A cluster can consist of multiple data nodes and master nodes to ensure balanced load distribution and resilience.

Elasticsearch Basics – Shards and Replicas

📤 **Shards:**

- An **index** is divided into **primary shards**, which store the documents. Each shard can be hosted on any node in the cluster, allowing parallel search across multiple nodes.

🔁 **Replicas:**

- **Replica shards** are copies of primary shards. They ensure **data redundancy** and help increase **query performance** by distributing the load.

🔧 **Sharding Benefits**

- **Scalability**: More primary shards allow the system to handle more data.
- **Fault Tolerance**: Replicas ensure data is available even if one node fails.

💡 **Example:**
If you have 3 primary shards and 2 replicas, Elasticsearch will maintain 6 copies of the data in total, distributed across the cluster for redundancy.

Elasticsearch Use Cases

🔍 **Full-Text Search**

- Elasticsearch is most commonly used for **full-text search** on large datasets. It allows applications to **search** and **filter** through unstructured text at high speeds.

📊 **Data Analytics**

- Elasticsearch is used for **log analysis**, **real-time monitoring**, and **business metrics analysis**. It's ideal for searching through massive logs and structured data.

💡 **Common Use Cases**

- **E-Commerce**: Search through products, reviews, and catalogs to help users find what they need.
- **Security**: Analyze security logs and detect intrusions or anomalies in real-time.
- **Real-Time Monitoring**: Collect and analyze performance metrics for infrastructure and applications.

Verify Elasticsearch Installation and Basic Cluster Information

Objective:
Ensure Elasticsearch is running and check cluster details.

Step 1: Start Elasticsearch

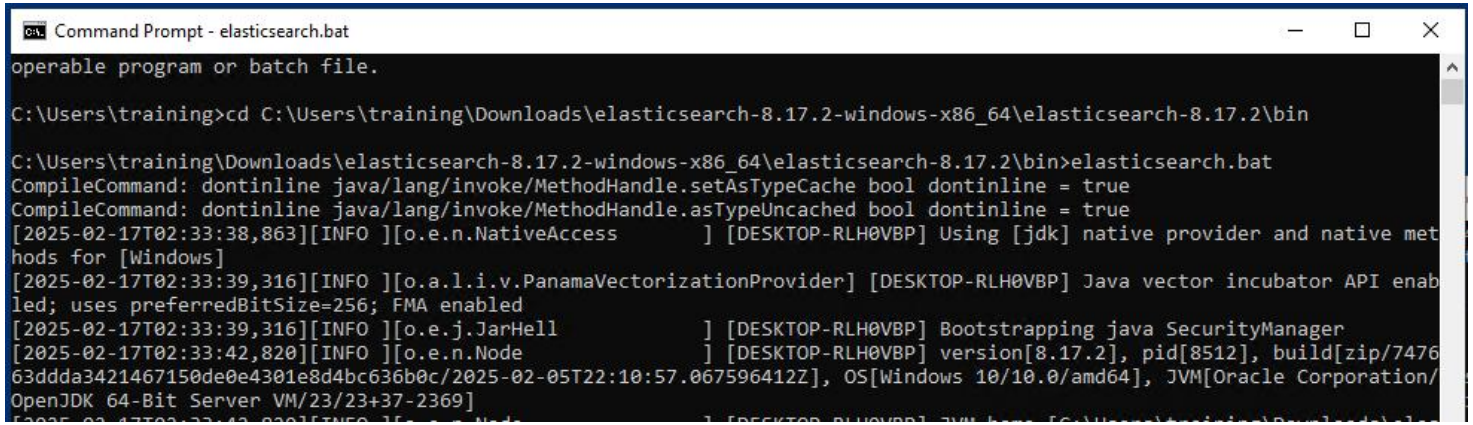
1. Open **Command Prompt** and navigate to the bin folder:

   cd C:\elasticsearch\bin

2. Start Elasticsearch:

   elasticsearch.bat

3. Keep this window open.



Verify Elasticsearch Installation and Basic Cluster Information

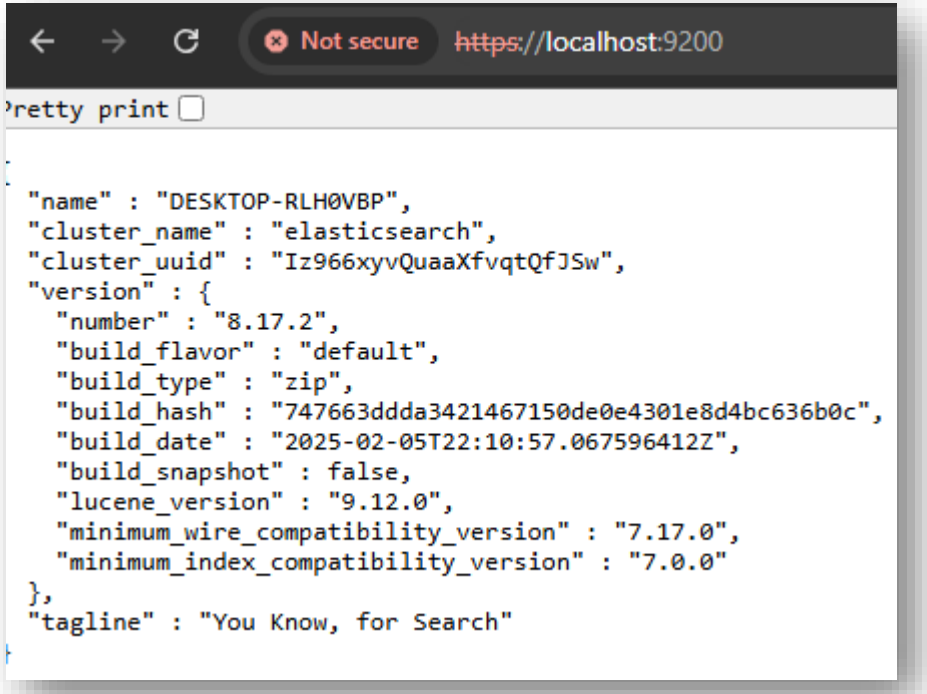
Step 2: Check If Elasticsearch is Running

1. Open a browser and go to:

https://localhost:9200

2. If Elasticsearch is running, you should see a JSON response like this: ----------→

3. If you see this, Elasticsearch is **successfully installed**.



Verify Elasticsearch Installation and Basic Cluster Information

Step 3: Check Cluster Health
Run the following command in a new **Command Prompt** window:

```
curl -X GET "https://localhost:9200/_cluster/health?pretty" --insecure -u elastic
```

💡 **Expected Output:**

- "status": "green" → Everything is working fine.
- "status": "yellow" → No replica assigned (expected with a single node).
- "status": "red" → Some primary shards are unassigned (**needs fixing**).

🎯 **Success!** You have verified your **Elasticsearch installation and cluster health.**

Verify Elasticsearch Installation and Basic Cluster Information

Command Prompt

```
}

C:\Users\training>curl -X GET "https://localhost:9200/_cluster/health?pretty" --insecure -u elastic
Enter host password for user 'elastic':
{
  "cluster_name" : "elasticsearch",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 31,
  "active_shards" : 31,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "unassigned_primary_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}

C:\Users\training>
```

Create Your First Index and Understand Indexing

Objective:

Learn how to create an index and understand how Elasticsearch stores data.

Step 1: Create a Simple Index

Run the following command to create an index called my_first_index:

curl -X PUT "https://localhost:9200/my_first_index" --insecure -u elastic

💡 **Expected Output:**

```
C:\Users\training>curl -X PUT "https://localhost:9200/my_first_index" --insecure -u elastic
Enter host password for user 'elastic':
{"acknowledged":true,"shards_acknowledged":true,"index":"my_first_index"}
C:\Users\training>
```

This confirms that the index is successfully created.

Create Your First Index and Understand Indexing

Step 2: View All Indices

To check if my_first_index is created, run:

curl -X GET "https://localhost:9200/_cat/indices?v" --insecure -u elastic

💡 **Expected Output:**

```
C:\Users\training>curl -X GET "https://localhost:9200/_cat/indices?v" --insecure -u elastic
Enter host password for user 'elastic':
health status index                                                    uuid                   pri rep docs.cou
nt docs.deleted store.size pri.store.size dataset.size
green  open   .internal.alerts-transform.health.alerts-default-000001  rs-XtGJnTYGoncBILvvx-w   1   0
 0            0       249b           249b         249b
yellow open   my_first_index                                           i5rfrtPrQ1Owo8SN_bJRYA   1   1
 0            0       227b           227b         227b
green  open   .internal.alerts-observability.logs.alerts-default-000001  V1N6r4SmSOi1ObrAKc_59A   1   0
```

- "yellow" is **expected** because we are running a **single-node** cluster (replica shards are unassigned).
- pri (primary shards) = **1**
- rep (replica shards) = **1**

🎯 **Success!** You have created your first Elasticsearch index.

theknowledgeacademy

Module 2:

Install, Manage, and Secure

- ✓ Set up Elasticsearch
- ✓ Secure the Elastic Stack
- ✓ Upgrade Elasticsearch
- ✓ Set up a Cluster for High Availability
- ✓ Monitor a Cluster
- ✓ Troubleshooting
- ✓ Enterprise Search Server

Setting Up Elasticsearch – Introduction

🚀 **What is Setup?**

- **Setting up Elasticsearch** involves installing the software, configuring the cluster, and ensuring it is ready for use.
- It's crucial to consider factors like **node configuration**, **cluster setup**, and **security** during the initial setup process.

🔧 **Elasticsearch Versions**

- Choose the right version based on system requirements, compatibility with plugins, and support for your use case.
- Use the **Elastic website** for installation packages and instructions specific to your environment.

Setting Up Elasticsearch – Introduction

🖥️ **Prerequisites**

- Ensure your system meets the necessary requirements, including **Java 11+**, sufficient memory (8GB+ recommended), and enough disk space.

🔑 **Key Considerations**

- **Security**: Implement security during setup (using **Elastic Security** for user authentication, encryption, etc.)
- **Networking**: Properly configure ports and firewalls.

Setting Up Elasticsearch – Installation Process

📥 **Installation Methods**

- **Download and Install**: Download the Elasticsearch package directly from the official website or through package managers (e.g., apt for Ubuntu, yum for CentOS).

Docker: Use a **Docker container** to run Elasticsearch for an easy and isolated environment setup.

- ⚙️ **Installation Steps**
  1. **Download the package** suitable for your OS.
  2. **Install** using command line or GUI-based package manager.
  3. **Configure Elasticsearch** (modify elasticsearch.yml for node and cluster settings).
  4. **Start the service** and ensure it is running by using curl to hit localhost:9200.

Setting Up Elasticsearch – Installation Process

📥 **Download the Packages and Unzip them**

1 Download and unzip Elasticsearch

Choose platform:

Windows

Windows sha asc

Package managers:

yum, dnf, or zypper apt-get

Containers:

Docker →

1 Download and unzip Kibana

Choose platform:

Windows

Windows sha asc

Package managers:

yum, dnf, or zypper apt-get

Containers:

Docker →

1 Download and unzip Logstash

Choose platform:

Windows

Windows sha asc

Package managers:

yum apt-get

Containers:

Docker →

Setting Up Elasticsearch – Installation Process

Steps to Install ElasticSearch and Kibana

1. Open up the elasticsearch-8.6.2 folder and within the config folder open the elasticsearch.yml file

2. Add this line into the file

```
action.auto_create_index: .monitoring*,.watches,.triggered_watches,.watcher-history*,.ml*
```

3. Your file will look like this

```
# Please consult the documentation for further information on configuration options:
# https://www.elastic.co/guide/en/elasticsearch/reference/index.html
#
action.auto_create_index: .monitoring*,.watches,.triggered_watches,.watcher-history*,.ml*

# ---------------------------------- Cluster -----------------------------------
```

4. Start the elasticsearch.bat file in the bin folder.

```
.\bin\elasticsearch.bat
```

Setting Up Elasticsearch – Installation Process

Steps to Install ElasticSearch and Kibana

5. Once you see the Token press CTRL + C and stop it and paste TOKEN into an empty notepad

```
The generated password for the elastic built-in superuser is:
<password>

The enrollment token for Kibana instances, valid for the next 30 minutes:
<enrollment-token>

The hex-encoded SHA-256 fingerprint of the generated HTTPS CA DER-encoded certificate:
<fingerprint>
```

6. Open elasticsearch.yml file again and scroll to the bottom and make these changes

```
# Enable security features
xpack.security.enabled: true

xpack.security.enrollment.enabled: true

# Enable encryption for HTTP API client connections, such as Kibana, Logstash, and Agents
xpack.security.http.ssl:
  enabled: false
  keystore.path: certs/http.p12

# Enable encryption and mutual authentication between cluster nodes
xpack.security.transport.ssl:
  enabled: false
  verification_mode: certificate
  keystore.path: certs/transport.p12
  truststore.path: certs/transport.p12
```

Setting Up Elasticsearch – Installation Process

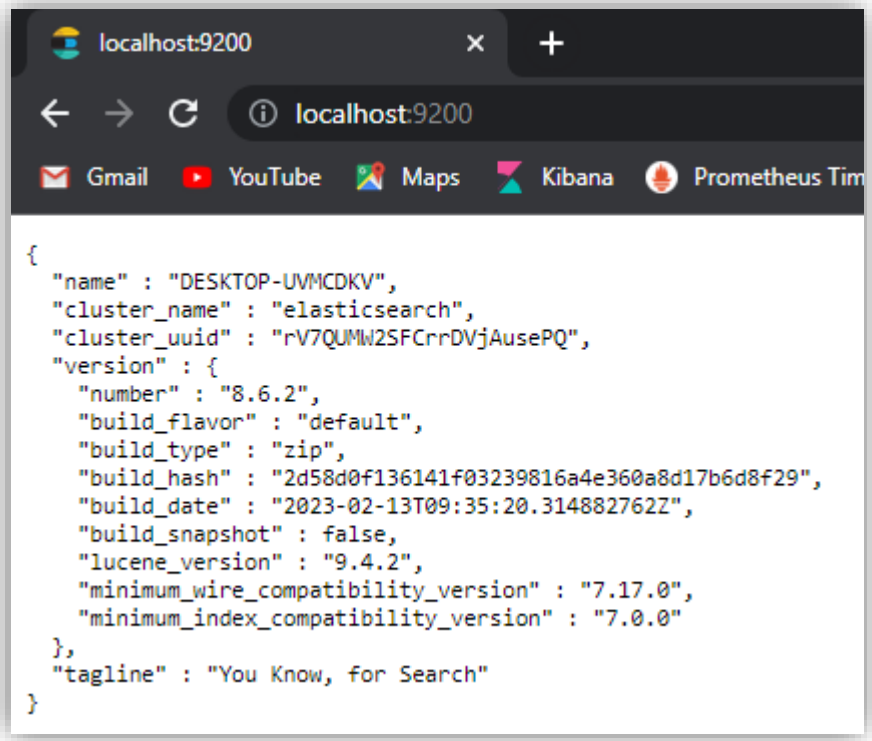
Steps to Install ElasticSearch and Kibana

7. Run elasticsearch.bat in cmd
8. After finishing the installation, open up **localhost:9200**
9. You will see this at the display
10. Reset passwords for Elastic and Kibana system

```
GO TO BIN directory and OPEN CMD

elasticsearch-reset-password -u elastic

elasticsearch-reset-password -u kibana_system
```



Setting Up Elasticsearch – Installation Process

Steps to Install Kibana

1. Start the kibana.bat file in the bin folder.

```
.\bin\kibana.bat
```

2. Make sure that the cmd prompt (Elasticsearch should be running in the other cmd prompt)

3. When the cmd prompt finishes, you need to go to a web tab page and type

https://localhost:5601

```
# If your Elasticsearch is protected with basic authentication, these settings provide
# the username and password that the Kibana server uses to perform maintenance on the Kibana
# index at startup. Your Kibana users still need to authenticate with Elasticsearch, which
# is proxied through the Kibana server.
#elasticsearch.username: "kibana_system"
#elasticsearch.password: "kCBxOuT3xGXvq3ZA51kM"
```

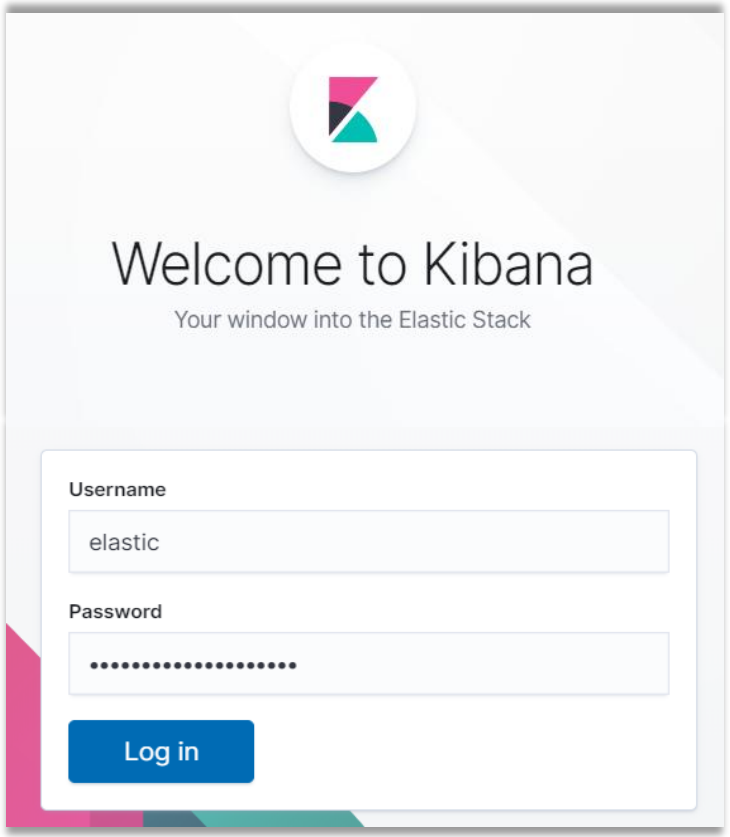
theknowledgeacademy

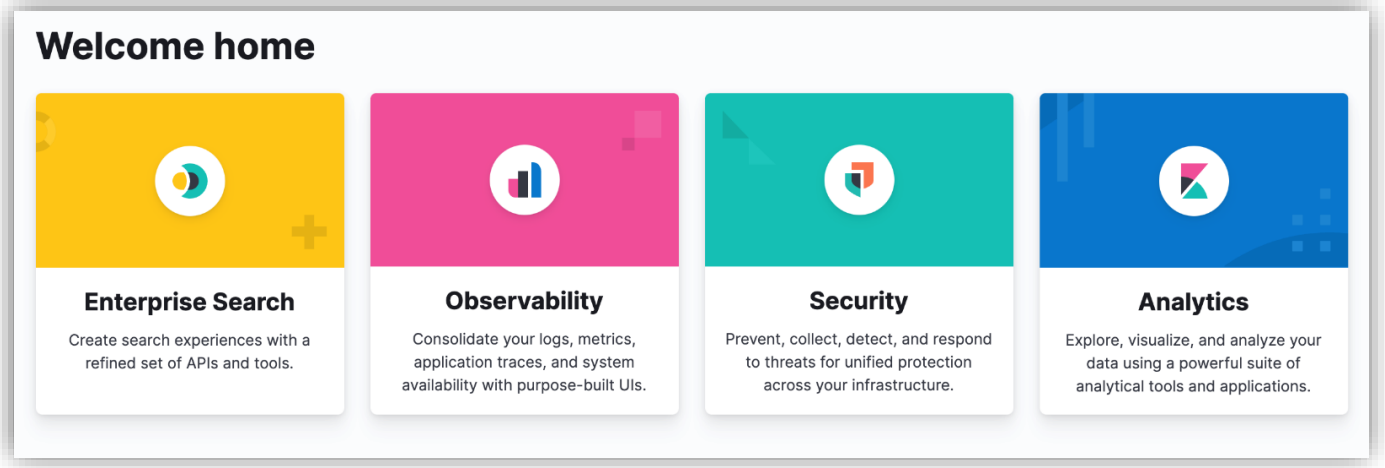
Setting Up Elasticsearch – Installation Process

Steps to Install Kibana

4. You will be taken to a login page of Kibana, where you need to fill in the detail of the elastic username and password that you defined in the Elasticsearch installation.

5. After filling in the details you will be taken to the Kibana console.





theknowledgeacademy

Setting Up Elasticsearch – Installation Process

Steps to Install Kibana

1. Start the logstash.bat file in the Logstash bin folder.

```
\bin>logstash.bat
```

2. Logstash will be installed

3. Else you can just Unzip the zip package and the Logstash is installed.

Securing the Elastic Stack – Introduction

🔒 Importance of Security

- Securing Elasticsearch is crucial for ensuring that sensitive data is protected from unauthorized access. The Elastic Stack includes built-in security features that allow encryption, authentication, and authorization.

🔑 Authentication & Authorization

- **Elastic Security** enables you to authenticate users and define access levels (roles, privileges).
- **Role-Based Access Control (RBAC)** lets you assign permissions based on roles.

🛡️ Encryption

- **TLS/SSL** encryption ensures that data transferred between nodes and clients is protected.
- Use **HTTPS** for encrypted communication to and from Elasticsearch nodes.

Securing the Elastic Stack – Users and Roles

👤 User Authentication

- Use **built-in user authentication** or integrate with external services like **LDAP**, **Active Directory**, or **SAML** for centralized user management.

🔒 Role-Based Access Control (RBAC)

- Assign specific roles to users that dictate the level of access and actions allowed within Elasticsearch.
- Common roles: **superuser**, **admin**, **read-only**, **indexer**.

🔑 Managing Permissions

- Define roles and permissions through **Elasticsearch's REST API** or by using **Kibana**'s graphical interface.
- Example: A user with the read role can only search data, while the write role can add or modify documents.

Securing the Elastic Stack – Encryption

🔐 TLS/SSL Encryption

- **Encrypt** data in transit using **Transport Layer Security (TLS)**.
- Ensure that **communications between nodes** in a cluster are secured.

🔒 Encrypting Data at Rest

- Encrypt sensitive data stored in **Elasticsearch indices** using the **Elastic Stack security features**.
- Implement encryption at the file system level for better protection.

🛡️ Authentication Mechanisms

- Elasticsearch provides **basic authentication** using **username and password**.
- For advanced scenarios, you can integrate with external **identity providers** using **SAML** or **OAuth**.

Upgrading Elasticsearch – Introduction

🔄 Importance of Upgrades

- Upgrading Elasticsearch ensures that you have the latest **features**, **security patches**, and **performance improvements**.

⚠️ Upgrade Planning

- **Test** the upgrade in a **staging environment** before applying it to production.
- Review **Elasticsearch's release notes** for breaking changes and deprecated features.

🚨 Backup Before Upgrade

- Always **backup your indices** and configuration files before starting the upgrade process to avoid data loss in case of issues.

Upgrading Elasticsearch – Upgrade Process

🛠️ **Step-by-Step Upgrade Process**

1. **Backup** Elasticsearch indices and configurations.
2. **Download the latest version** from the official site.
3. **Install the update** using your preferred method (e.g., package manager, Docker, etc.).
4. **Verify version compatibility** between your existing and upgraded clusters.
5. **Restart** Elasticsearch nodes, and check for any issues using logs.

⚙️ **Rolling Upgrades**

- **Rolling upgrades** allow you to upgrade clusters **node by node** to avoid downtime.
- Ensure compatibility between old and new nodes during the upgrade process.

💡 **Tip:**

- Consider using **Elastic Cloud** for managed Elasticsearch, which handles upgrades automatically.

theknowledgeacademy

Setting Up a Cluster for High Availability – Introduction

🌐 **Cluster Overview**

- A **cluster** is a group of nodes working together to share data and provide fault tolerance.
- Ensure that Elasticsearch remains available even if a **node** fails by setting up **replication** and **sharding** strategies.

📡 **High Availability**

- High availability (HA) ensures that Elasticsearch services continue operating even if one or more nodes fail.
- Use **multiple master nodes** to manage the cluster and **replica shards** to ensure data redundancy.

🔄 **Cluster Recovery**

- Elasticsearch automatically rebalances data and recovers from node failures by using replica shards.

Setting Up a Cluster for High Availability – Cluster Configuration

⚙️ **Cluster Sizing**

- Consider **hardware resources** (RAM, CPU, storage) when configuring the number of nodes and shard replicas.
- **Geographically distributed clusters** help improve fault tolerance in case of network failures or outages.

🔑 **Key Cluster Settings**

- Define **number of nodes** based on the scale of your deployment.
- Set **discovery.zen.minimum_master_nodes** to avoid split-brain scenarios where two parts of the cluster can't communicate.

💡 **Load Balancing**

- Use a **load balancer** to evenly distribute client requests across nodes in a cluster, preventing any single node from becoming overwhelmed.

Monitoring a Cluster – Introduction

📊 **Importance of Monitoring**

- Monitoring Elasticsearch clusters is essential for ensuring **performance**, **availability**, and **security**.
- Use tools like **Kibana** and **Elastic APM** to gain insights into cluster health and performance.

🔍 **Key Metrics to Monitor**

- Monitor **CPU usage**, **memory usage**, **disk space**, **query performance**, and **shard distribution**.
- Use **cluster health** APIs to assess cluster status (green, yellow, red).

🔧 **Preventive Actions**

- Set up **alerts** for critical thresholds like high CPU usage or low disk space to proactively address issues.

Monitoring a Cluster – Tools and Dashboards

📊 **Kibana Monitoring**

- Kibana offers a **Monitoring UI** to visualize key metrics like cluster health, node status, and query performance.
- View detailed logs, shard status, and search latency.

💻 **Elastic Stack Monitoring**

- **Elastic Agent** and **Beats** can be used to collect and forward monitoring data to your central Elasticsearch cluster.
- Elastic APM (Application Performance Monitoring) tracks application performance, making it easier to spot bottlenecks.

🔔 **Alerting**

- Use **Watcher** or **Elasticsearch Alerts** to set up custom notifications for issues like high latency or disk space problems.

theknowledgeacademy

Troubleshooting Elasticsearch

🛠️ **Common Elasticsearch Issues**

- Performance degradation, query slowdowns, and node failures are common problems.
- Troubleshooting involves **log analysis**, **cluster health checks**, and **resource monitoring**.

🔍 **Common Causes of Problems**

- **Resource bottlenecks**: CPU, memory, or disk I/O.
- **Cluster misconfigurations**: Issues with node discovery, shard allocation, and network settings.
- **Data issues**: Corruption, improper mappings, or insufficient disk space.

⚠️ **Tools for Troubleshooting**

- Use **Elasticsearch logs**, **node stats APIs**, and **Kibana** for troubleshooting insights.

Troubleshooting Elasticsearch

📈 **Query Performance Tuning**

- **Optimize queries** by ensuring proper indexing, using **filters** and **aggregations** effectively, and using **doc values** for field data.

📚 **Elasticsearch Logs**

- Review **Elasticsearch logs** for errors related to node failures, shard allocation issues, or slow queries.
- Enable **debug logs** for more detailed information.

🛠️ **Rebalancing Shards**

- **Manually rebalance** shards if Elasticsearch fails to distribute them evenly across nodes.
- Use **allocation filters** to control where shards should reside.

Enterprise Search Server

💼 **What is an Enterprise Search Server?**

- An **Enterprise Search Server** is a solution that provides scalable, efficient search capabilities for organizations across multiple data sources.
- It leverages Elasticsearch as the core engine to provide **advanced search features**, such as **full-text search**, **faceting**, and **filtering**.

📚 **Benefits of Enterprise Search**

- Allows employees to search internal knowledge bases, documents, emails, and other data sources efficiently.
- Improves productivity by providing fast, relevant search results for users.

Enterprise Search Server

🔑 **Key Features**

- **Federated Search**: Search across multiple data sources.
- **Customizable** search UI for users.
- **Real-time data indexing** for immediate searchability.

theknowledgeacademy

Enterprise Search Server – Setup and Use

⚙️ **Setting Up Enterprise Search**

- Install **Elastic Enterprise Search** on your infrastructure.
- Configure connectors for different data sources, like **G Suite**, **SharePoint**, or **Amazon S3**.

🔌 **Connecting Data Sources**

- Use **pre-built connectors** or create custom connectors to integrate various platforms with Elasticsearch.
- Data can be indexed in **real-time** or on a schedule.

🚀 **Scaling Enterprise Search**

- Scale **search performance** by distributing the load across multiple **Elasticsearch nodes**.
- Optimize the search experience with **caching** and **shard allocation** strategies.

theknowledgeacademy

Module 3:

Ingest Data

- ✓ Adding Data to Elasticsearch
- ✓ Connectors
- ✓ Web Crawler
- ✓ Data Streams
- ✓ Ingest Pipelines
- ✓ Mapping

Introduction to Data Ingestion in Elasticsearch

📥 **What is Data Ingestion?**

- Ingesting data into Elasticsearch is the process of adding various types of data (structured, unstructured, logs, etc.) for indexing and search.
- Data must be properly processed and formatted before it can be indexed.

🔄 **Types of Data Sources**

- Elasticsearch can ingest data from multiple sources, including **databases**, **logs**, and **third-party applications**.
- Common data formats: **JSON**, **CSV**, **XML**, and **log files**.

🚀 **Data Flow Overview**

- Data typically flows through an ingestion process before reaching Elasticsearch's index for fast search and analysis.

Methods of Data Ingestion in Elasticsearch

🧮 API Ingestion

- You can use Elasticsearch's **RESTful API** to send data directly to the server.
- This is ideal for **real-time data** ingestion.

🔄 Bulk Ingestion

- **Bulk API** is used for efficiently adding large amounts of data.
- It's suitable for **batch processing**, allowing multiple documents to be indexed in a single request.

🔧 Logstash Ingestion

- **Logstash** is a data processing pipeline that can ingest data from various sources, transform it, and load it into Elasticsearch.
- It supports **real-time streaming** and **batch processing**.

Introduction to Connectors in Elasticsearch

🔌 **What are Connectors?**

- **Connectors** allow Elasticsearch to easily integrate with external data sources like **databases**, **cloud storage**, and **SaaS applications**.
- They provide a seamless way to index and manage external data directly within Elasticsearch.

🧩 **Types of Connectors**

- **Beats**: Lightweight data shippers to send log and metric data to Elasticsearch.
- **Logstash**: Used for gathering, parsing, and sending data from various sources to Elasticsearch.
- **Elasticsearch connectors** for various third-party applications like **JDBC**, **SharePoint**, and more.

📂 **Use Cases for Connectors**

- Connecting to **relational databases**, **cloud platforms**, and **file systems** for automatic indexing of external data.

Types of Connectors – Beats

🎯 **Beats Overview**

- Beats are lightweight data shippers that collect and ship data directly to Elasticsearch or Logstash.
- **Filebeat**: For log data.
- **Metricbeat**: For system and application metrics.
- **Packetbeat**: For network data.

🔄 **How Beats Work**

- Beats send **data in real-time** from various sources like servers, applications, and logs to Elasticsearch.
- Beats can be configured to ship data at regular intervals for **continuous monitoring**.

💡 **Key Benefits**

- **Lightweight**: Minimal resource overhead.
- **Real-time data shipping**: Enables immediate indexing of new data.

Using Connectors for Data Ingestion

🔌 **How to Use Connectors**

- Set up connectors in **Kibana** or **Logstash** to connect your external data sources.
- Define the data types (e.g., relational, unstructured) and create mappings for indexing.

🛠️ **Configuration Steps**

- Configure your **connector settings** (e.g., for **JDBC** or **cloud platforms**).
- Use **Logstash pipelines** to process and ingest data from the connector into Elasticsearch.

🔄 **Real-Time Data Ingestion**

- Once set up, connectors ensure that data is continuously ingested and updated in Elasticsearch in **real-time**.

Introduction to Web Crawlers

🌐 **What is a Web Crawler?**

- A **web crawler** is a tool that automatically searches and collects data from websites, enabling you to index external web content in Elasticsearch.
- It helps in **search engine optimization**, data extraction, and **web scraping**.

🔍 **Use Cases for Web Crawlers**

- Indexing **public websites** for searchability.
- **Scraping content** for analysis (e.g., news articles, blogs).

⚙️ **How It Works**

- Web crawlers visit URLs, extract data, and send it to Elasticsearch for indexing.

Using a Web Crawler to Index External Data

🔧 **Setting Up the Crawler**

- Configure the **crawler's URL list**: Provide the starting URLs for the crawler to begin collecting data.
- **Data Mapping**: Define how the data will be structured in Elasticsearch once ingested.

📝 **Extracting Relevant Data**

- Crawlers can extract **text**, **metadata**, and **links** from web pages.
- Define the data fields and decide what to index (e.g., **title**, **author**, **publish date**).

📡 **Continuous Crawling**

- Set crawlers to run **periodically** to index new data from web pages.

What are Data Streams

💡 **Introduction to Data Streams**

- **Data streams** allow you to manage time-series data more efficiently.
- They are ideal for handling **continuous data** (e.g., logs, sensor data, financial transactions).

🔄 **Key Features of Data Streams**

- Automatically **rotate** data when it becomes outdated.
- Optimized for **real-time ingestion** of continuous events.

🎯 **When to Use Data Streams?**

- For **real-time log data**, **metrics**, and **event-based data**.

Configuring and Managing Data Streams

⚙️ **Data Stream Configuration**

- Define the **index template** for time-series data.
- Set up **write indices** that automatically rotate and manage the data.

🗓️ **Rolling Over Data**

- Configure **index rollover policies** to manage old data and prevent performance degradation.

🔧 **Optimizing Data Streams**

- Fine-tune settings like **sharding** and **replication** to ensure performance at scale.

Introduction to Ingest Pipelines

🔧 **What are Ingest Pipelines?**

- An **ingest pipeline** is a series of data transformation steps before data is indexed into Elasticsearch.
- Ingest pipelines can perform tasks like **data parsing**, **field mapping**, and **enrichment**.

⚙️ **Key Functions of Ingest Pipelines**

- **Processor** components: Handle data transformations like **grok**, **date**, or **geoip** processors.
- Transform and enrich data before indexing.

🌟 **Use Cases for Ingest Pipelines**

- Parsing logs, enriching data from external sources, and transforming raw data into a structured format.

Creating and Configuring an Ingest Pipeline

🛠️ **Configuring Processors**

- Choose the right **processor** for your data (e.g., **grok** for parsing, **set** for field values).
- Combine multiple processors in a pipeline for complex data transformations.

- 🎯 **How to Define a Pipeline**
  - Use the **PUT pipeline API** to define your pipeline's steps.
  - Example:

```
PUT _ingest/pipeline/my_pipeline
{
  "description": "Parse log data",
  "processors": [
    {
      "grok": {
        "field": "message",
        "patterns": ["%{COMMONAPACHELOG}"]
      }
    }
  ]
}
```

Processing and Enhancing Data through Pipelines

🔄 **Enhancing Data**

- Ingest pipelines can enrich data by adding external fields (e.g., geo-location, IP data) using processors like **geoip** or **user-agent**.

🔧 **Transforming Data**

- Convert data to the desired format, e.g., parsing **JSON** logs into key-value pairs.

⚙️ **Common Pipeline Processors**

- **Grok**: Parses unstructured data.
- **Date**: Converts string dates into proper **date formats**.
- **Set**: Adds or modifies field values.

Common Use Cases for Ingest Pipelines

📊 **Use Case 1: Log Parsing**

- Ingest pipelines can be used to **parse logs** and convert them into structured data.

📡 **Use Case 2: Real-time Metrics**

- Data from **metrics** sources can be processed in real time using pipelines for immediate indexing.

➢ **Use Case 3: Data Enrichment**

- External data like **geo-location** and **user agent** information can be added using processors to enhance the data quality.

Introduction to Mappings and Data Types

🔄 **What Are Mappings?**

- **Mappings** define how Elasticsearch indexes and stores data in **fields**.
- It specifies how different **data types** (e.g., text, integer, date) are handled by Elasticsearch.

🎯 **Purpose of Mappings**

- To structure your data and improve **search accuracy** and **performance**.
- Helps in defining **how data should be stored** and **how fields are analyzed** (e.g., text fields can be analyzed for full-text search).

🌐 **Dynamic vs. Custom Mappings**

- **Dynamic Mappings**: Automatically detected by Elasticsearch when documents are indexed.
- **Custom Mappings**: Defined by the user for specific needs (e.g., changing field types, adding analyzers).

Understanding Data Types in Mappings

🔢 Key Data Types in Elasticsearch

- **Text**: Suitable for full-text search, where data is tokenized.
- **Keyword**: For exact matches, e.g., tags, IDs, etc.
- **Integer/Long**: For numeric fields (e.g., quantity, price).
- **Date**: For storing dates in various formats (e.g., yyyy-MM-dd).
- **Boolean**: For true/false values.

⚙️ Data Type Importance

- **Correct Data Type Mapping** improves search efficiency and data handling.
- **Incorrect data types** can cause **search errors** or data interpretation issues.

📝 Example Mapping

```
"properties": {
 "name": {
  "type": "text"
 },
 "price": {
  "type": "float"
 },
 "release_date": {
  "type": "date"
 }
}
```

Hands-on 1: Add Sample Data to Elasticsearch

Objective:
Learn how to **index (store) data** in Elasticsearch using REST API.

Step 1: Create an Index (employees)
Run this command in **Command Prompt**:

```
curl -X PUT "https://localhost:9200/employees" --insecure -u elastic:<your_password>
```

💡 **Expected Output:**

```
C:\Users\training>curl -X PUT "https://localhost:9200/employees" --insecure -u elastic
Enter host password for user 'elastic':
{"acknowledged":true,"shards_acknowledged":true,"index":"employees"}
C:\Users\training>
```

Hands-on 1: Add Sample Data to Elasticsearch

Step 2: Insert a Document (Employee Data)
Now, let's add a sample employee record:

```
curl -X POST "https://localhost:9200/employees/_doc/1" --insecure -u elastic:<your_password> -H "Content-Type: application/json" -d ^ "{ \"name\": \"John Doe\", \"age\": 30, \"position\": \"Software Engineer\", \"salary\": 75000 }"
```

💡 **Expected Output:**

```
C:\Users\training>curl -X POST "https://localhost:9200/employees/_doc/1" --insecure -u elastic:hazIltymKVQeCoVKzlID -H "Content-Type: application/json" -d ^
More? "{\"name\": \"John Doe\", \"age\": 30, \"position\": \"Software Engineer\", \"salary\": 75000}"
{"_index":"employees","_id":"1","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":0,"_primary_term":1}
C:\Users\training>
```

🎯 **Success!** You have indexed your first document in Elasticsearch.

Hands-on 1: Add Sample Data to Elasticsearch

Step 3: Retrieve the Inserted Data
Now, retrieve the data from the employees index:

curl -X GET "https://localhost:9200/employees/_doc/1" --insecure -u elastic:<your_password>

💡 **Expected Output:**

```
C:\Users\training>curl -X GET "https://localhost:9200/employees/_doc/1" --insecure -u elastic:hazIltymKVQeCoVKzlID
{"_index":"employees","_id":"1","_version":1,"_seq_no":0,"_primary_term":1,"found":true,"_source":{"name": "John Doe", "age": 30, "position": "Software Engineer", "salary": 75000}}
C:\Users\training>
```

Hands-on 1: Add Sample Data to Elasticsearch

Step 4: Retrieve All Documents in the Index
To list all documents stored in the employees index:

curl -X GET "https://localhost:9200/employees/_search?pretty" --insecure -u elastic:<your_password>

💡 **Expected Output:**

```
C:\Users\training>curl -X GET "https://localhost:9200/employees/_search?pretty" --insecure -u elastic:hazIltymKVQeCoVKzlID
{
  "took" : 98,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
      "value" : 1,
      "relation" : "eq"
    },
    "max_score" : 1.0,
    "hits" : [
      {
        "_index" : "employees",
        "_id" : "1",
        "_score" : 1.0,
        "_source" : {
          "name" : "John Doe",
          "age" : 30,
          "position" : "Software Engineer",
          "salary" : 75000
        }
      }
    ]
  }
}

C:\Users\training>_
```

✅ This confirms that Elasticsearch **contains the expected documents**.

Hands-on 2: Define and Use Mappings

🎯 **Objective:**

Learn how to **define custom field types** in Elasticsearch using mappings.

- We will create an index called products with predefined **data types** for structured storage.
- We will then insert a document and verify that Elasticsearch **correctly enforces the mappings**.

✅ **Step 1: Create an Index with Custom Mappings**

Run this command in **Windows Command Prompt (CMD)**:

```
curl -X PUT "https://localhost:9200/products" --insecure -u elastic:<your_password> -H "Content-Type: application/json" -d ^
"{\"mappings\": {\"properties\": {\"name\": {\"type\": \"text\"},\"price\": {\"type\": \"float\"},\"category\": {\"type\": \"keyword\"},\"availability\": {\"type\": \"boolean\"}}}}"
```

Hands-on 2: Define and Use Mappings

💡 **Expected Output:**

```
C:\Users\training>curl -X PUT "https://localhost:9200/products" --insecure -u elastic:hazIltymKVQeCoVKzlID -H "Content-Type: application/json" -d ^
More? "{\"mappings\": {\"properties\": {\"name\": {\"type\": \"text\"},\"price\": {\"type\": \"float\"},\"category\": {\"type\": \"keyword\"},\"availability\": {\"type\": \"boolean\"}}}}"
{"acknowledged":true,"shards_acknowledged":true,"index":"products"}
C:\Users\training>
C:\Users\training>
```

🎯 **You have successfully created an index with structured data types!**

Hands-on 2: Define and Use Mappings

✅ **Step 2: Insert a Product Document**

Now, let's add a product to the products index:

```
curl -X POST "https://localhost:9200/products/_doc/1" --insecure -u elastic:<your_password> -H "Content-Type: application/json" -d ^ "{\"name\": \"Laptop\",\"price\": 1200.50,\"category\": \"Electronics\",\"availability\": true}"
```

💡 **Expected Output:**

```
C:\Users\training>curl -X POST "https://localhost:9200/products/_doc/1" --insecure -u elastic:hazIltymKVQeCoVKzlID -H "Content-Type: application/json" -d ^
More? "{\"name\": \"Laptop\",\"price\": 1200.50,\"category\": \"Electronics\",\"availability\": true}"
{"_index":"products","_id":"1","_version":1,"result":"created","_shards":{"total":2,"successful":1,"failed":0},"_seq_no":0,"_primary_term":1}
C:\Users\training>
```

🎯 **Now you have structured data stored in Elasticsearch!**

Hands-on 2: Define and Use Mappings

✅ **Step 3: Verify the Mapping of the Index**

To check whether Elasticsearch correctly stored the mappings, run:

```
curl -X GET "https://localhost:9200/products/_mapping?pretty" --insecure -u elastic:<your_password>
```

💡 **Expected Output:**

```
C:\Users\training>curl -X GET "https://localhost:9200/products/_mapping?pretty" --insecure -u elastic:hazIltymKVQeCoVKzlID
{
  "products" : {
    "mappings" : {
      "properties" : {
        "availability" : {
          "type" : "boolean"
        },
        "category" : {
          "type" : "keyword"
        },
        "name" : {
          "type" : "text"
        },
        "price" : {
          "type" : "float"
        }
      }
    }
  }
}

C:\Users\training>
```

🎯 **Elasticsearch has enforced the correct field types!**

theknowledgeacademy

Module 4:
Search and Analyse

- ✓ Search your Data
- ✓ Query Data with the Query DSL, ES|QL, EQL, or SQL
- ✓ Search Applications
- ✓ Search Analytics

Search your Data

❖ **Introduction to Searching in Elasticsearch**

Elasticsearch provides powerful search capabilities, enabling users to retrieve and analyze data efficiently. The search functionality is based on a distributed, scalable, and fast engine optimized for structured and unstructured data. By leveraging Elasticsearch's query features, users can perform full-text searches, aggregations, and filtering with high performance.

- Elasticsearch uses an inverted index for fast search performance.
- It supports structured, unstructured, and geospatial searches.
- Users can leverage query DSL, SQL, and other query languages for diverse search needs.
- Search results are ranked and scored based on relevance.
- The system supports real-time and near-real-time searching capabilities.

Search your Data

❖ **Basic Search Mechanism in Elasticsearch**

The search mechanism in Elasticsearch revolves around indexing and querying. When data is indexed, Elasticsearch structures it for fast retrieval. Queries can be simple keyword searches or complex multi-condition searches.

- Searching is performed using the _search endpoint.
- Queries can be full-text, term-based, or structured queries.
- Elasticsearch supports Boolean queries for combining multiple conditions.
- Filtering can be applied to refine results without affecting scores.
- Pagination is supported to handle large result sets efficiently.

Search your Data

❖ **Steps to carry out basic Search**

Step 1: Enter data into your index

```
curl -X POST "http://localhost:9200/library/_bulk" -H "Content-Type: application/json" -d'
{"index":{"_id":"1"}}
{"title":"Elasticsearch Essentials", "author":"John Doe", "year":2021}
{"index":{"_id":"2"}}
{"title":"Advanced Elasticsearch", "author":"Jane Smith", "year":2023}
{"index":{"_id":"3"}}
{"title":"Data Search Optimization", "author":"Emily Johnson", "year":2020}
'
```

You'll get output:

```
{
  "took": 30,
  "errors": false,
  "items": [...]
}
```

Search your Data

Step 2: Search for all books in your index

```
curl -X GET "http://localhost:9200/library/_search?pretty"
```

The output:


```
{
  "hits": {
    "total": 3,
    "hits": [
      {
        "_id": "1",
        "_source": {
          "title": "Elasticsearch Essentials",
          "author": "John Doe",
          "year": 2021
        }
      },
      {
        "_id": "2",
        "_source": {
          "title": "Advanced Elasticsearch",
          "author": "Jane Smith",
          "year": 2023
        }
      },
      {
        "_id": "3",
        "_source": {
          "title": "Data Search Optimization",
          "author": "Emily Johnson",
          "year": 2020
        }
      }
    ]
  }
}
```

Search your Data

Step 3: Search for a particular author

```
curl -X GET "http://localhost:9200/library/_search?pretty" -H "Content-Type: application/json" -d'
{
  "query": {
    "match": {
      "author": "John Doe"
    }
  }
}'
```

Output:

```
{
  "hits": {
    "total": 1,
    "hits": [
      {
        "_id": "1",
        "_source": {
          "title": "Elasticsearch Essentials",
          "author": "John Doe",
          "year": 2021
        }
      }
    ]
  }
}
```

theknowledgeacademy

Search your Data

❖ **Types of Search Queries**

Elasticsearch provides multiple types of queries to cater to different search needs. These queries determine how data is retrieved and ranked based on relevance.

- **Match Query** – Performs full-text searches with relevance scoring.
- **Term Query** – Looks for exact term matches in structured data.
- **Boolean Query** – Combines multiple queries using logical operators (AND, OR, NOT).
- **Range Query** – Filters results based on numerical or date-based conditions.
- **Wildcard and Regex Queries** – Allow pattern-based searching for flexible matches.

Search your Data

❖ **Full-Text Search in Elasticsearch**

Full-text search is one of the core features of Elasticsearch, allowing efficient and relevant document retrieval. It utilizes analyzers to break text into tokens, making searches more effective.

- Full-text search is powered by Elasticsearch's **analyzers and tokenizers**.
- The **match query** is used for full-text searches, considering synonyms and stemming.
- **Relevance scoring** is applied using the BM25 ranking algorithm.
- Searches support **fuzzy matching** to find results with minor spelling variations.
- Custom analyzers can be configured to optimize search behavior.

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ Introduction to Querying in Elasticsearch

Elasticsearch provides multiple query languages to retrieve and analyze data efficiently. Users can query data using **Query DSL, ES|QL, EQL, or SQL**, depending on the use case. Each language offers different capabilities, from structured searches to event-driven analytics.

❖ Key Query Languages in Elasticsearch:

- **Query DSL** – A JSON-based query language for full-text search and structured queries.
- **ES|QL (Elasticsearch Query Language)** – A pipeline-based language for transformations and analytics.
- **EQL (Event Query Language)** – Optimized for event-driven searches and security analytics.
- **SQL** – Enables querying Elasticsearch using traditional relational database syntax.
- **Use Cases** – Query DSL for complex searches, ES|QL for data processing, EQL for security, and SQL for relational users.

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **Understanding Query DSL in Elasticsearch**

Query DSL (Domain-Specific Language) is the primary method for querying data in Elasticsearch. It allows users to perform powerful searches using JSON-based syntax, supporting structured filtering and full-text search.

❖ **Types of Queries in Query DSL:**

- **Match Query** – Performs full-text searches on analyzed fields.
- **Term Query** – Retrieves exact matches for keyword-based searches.
- **Boolean Query** – Combines multiple conditions with must, should, must_not, and filter.
- **Range Query** – Filters results based on numerical or date-based conditions.
- **Aggregation Queries** – Performs statistical computations and data grouping.

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **How to Query Data Using Elasticsearch Query DSL (Match Query)**

Utilize the following code:

```
curl -X GET "http://localhost:9200/library/_search?pretty" -H "Content-Type: application/json" -d'
{
  "query": {
    "range": {
      "year": {
        "gte": 2021
      }
    }
  }
}'
```

Output:

```
{
  "hits": {
    "total": 2,
    "hits": [
      { "_source": { "title": "Elasticsearch Essentials", "year": 2021 }},
      { "_source": { "title": "Advanced Elasticsearch", "year": 2023 }}
    ]
  }
}
```

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **ES|QL – The Elasticsearch Query Language**

ES|QL (Elasticsearch Query Language) is a pipeline-based language for real-time data transformation and analytics. It enables users to apply sequential operations to filter, transform, and analyze data.

❖ **Features of ES|QL:**

- **Pipeline Processing** – Queries are structured as a sequence of transformation steps.
- **Advanced Filtering & Sorting** – Enables refined data retrieval.
- **Joins and Data Merging** – Supports combining datasets for deeper analysis.
- **Time-Series Analysis** – Ideal for monitoring logs and analyzing event-driven data.
- **Kibana Integration** – Works seamlessly with Kibana for visualization and dashboards

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **Steps to Query Data Using ES|QL**

Utilize the following Code

```
curl -X POST "http://localhost:9200/_query" -H "Content-Type: application/json" -d'
{
  "query": "FROM library | WHERE year > 2020 | SORT year DESC"
}'
```

And the output:

```
{
  "columns": ["title", "year"],
  "rows": [
    ["Advanced Elasticsearch", 2023],
    ["Elasticsearch Essentials", 2021]
  ]
}
```

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **EQL – Event Query Language for Security Analytics**

EQL (Event Query Language) is designed for event correlation, security analytics, and anomaly detection. It helps detect patterns in time-based datasets and log data.

❖ **Capabilities of EQL:**

- **Sequence Queries** – Identifies patterns across related events in a specific order.
- **Anomaly Detection** – Finds unusual behavior in logs and security events.
- **Security Use Cases** – Used in SIEM (Security Information and Event Management) for cyber threat detection.
- **Filtering Conditions** – Supports advanced filters like where, until, and by
- **Optimized Performance** – Designed for large-scale event datasets.

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **Steps to Query Using EQL (Event Query Language)**

Utilize the following Code

```
curl -X GET "http://localhost:9200/library/_eql/search" -H "Content-Type: application/json" -d'
{
  "query": "author == \"John Doe\""
}'
```

And the output:

```
{
  "hits": {
    "total": 1,
    "hits": [{ "_source": { "title": "Elasticsearch Essentials", "author": "John Doe" }}]
  }
}
```

Query Data with the Query DSL, ES|QL, EQL, or SQL

❖ **SQL Support in Elasticsearch**

Elasticsearch includes **SQL support**, allowing users to execute relational-style queries on indexed data. This feature helps bridge the gap between structured and unstructured searches.

❖ **SQL Capabilities in Elasticsearch:**

- **Standard Queries** – Supports SELECT, FROM, WHERE, GROUP BY, and ORDER BY.
- **Joins and Aggregations** – Enables advanced filtering and grouping operations.
- **SQL REST API & JDBC** – Allows integration with external applications and databases.
- **Translate API** – Converts SQL queries into Elasticsearch Query DSL for execution.
- **Kibana SQL Workbench** – Provides a UI for writing and testing SQL queries.

Search Applications

❖ **Introduction to Search Applications in Elasticsearch**

Search applications built on **Elasticsearch** leverage its powerful indexing and querying capabilities to deliver fast and relevant search results. These applications are widely used across industries for real-time data retrieval and analytics.

❖ **Key Aspects of Search Applications:**

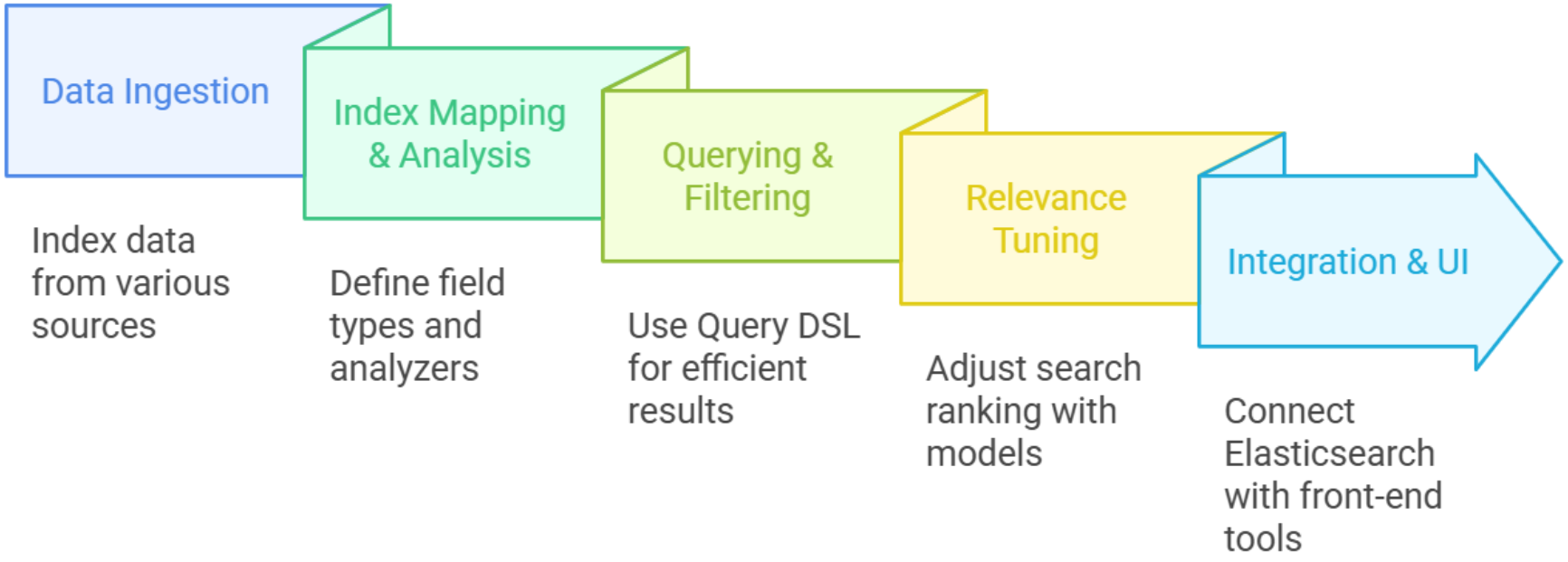
- **Full-Text Search** – Enables efficient searching across large text datasets.
- **Structured & Unstructured Data Support** – Handles both keyword-based and free-text queries.
- **Scalability** – Can handle massive datasets with distributed processing.
- **Real-Time Indexing** – Ensures up-to-date search results as data is added or modified.
- **Customization** – Allows tuning search relevance with ranking and boosting strategies.

Search Applications

❖ **Building a Search Application with Elasticsearch**

Elasticsearch provides various tools and APIs to build search applications tailored to specific business needs. The choice of implementation depends on use cases like e-commerce, document search, or log analysis.

❖ **Steps to Build a Search Application:**



theknowledgeacademy

Search Applications

❖ **Enhancing Search with Relevance Tuning**

Optimizing **search relevance** ensures users receive the most accurate and useful results. Elasticsearch provides several mechanisms to fine-tune search queries for improved ranking and relevance.

❖ **Techniques for Relevance Tuning:**

- **Boosting Fields & Queries** – Prioritize certain fields or specific terms in search results.
- **Custom Scoring Functions** – Use function score queries to influence ranking dynamically.
- **Synonyms & Stemming** – Expand search terms to include similar words or variations.
- **Phrase & Proximity Matching** – Improve ranking by considering word order and closeness.
- **User Behavior Analysis** – Adjust search relevance based on user engagement and click-through rates.

Search Applications

❖ **Using Machine Learning for Smart Search**

Machine learning can significantly enhance **search applications** by improving result accuracy, personalization, and intent recognition. Elasticsearch integrates with ML tools to refine search capabilities.

❖ **Machine Learning Applications in Search:**

- **Query Expansion** – Suggests alternative or related queries based on user behavior.
- **Anomaly Detection** – Identifies unusual search patterns and detects potential issues.
- **Personalized Ranking** – Adapts search results based on individual user preferences.
- **Semantic Search** – Understands user intent beyond exact keyword matching.
- **Auto-Completion & Predictions** – Suggests search terms as users type for a better experience.

Search Analytics

❖ Introduction to Search Analytics

Search analytics provides insights into how users interact with search systems, helping organizations optimize performance and improve user experience. Elasticsearch enables real-time search analytics through its powerful aggregation framework.

❖ Importance of Search Analytics:

- **User Behavior Analysis** – Tracks search patterns, click-through rates, and common queries.
- **Query Performance Monitoring** – Identifies slow queries and optimizes indexing strategies.
- **Relevance Assessment** – Evaluates search result accuracy and ranking effectiveness.
- **Trend Detection** – Analyzes historical search trends to predict future needs.
- **Personalization & Optimization** – Enhances search results based on data-driven insights.

Search Analytics

❖ **Using Aggregations for Search Analytics**

Elasticsearch **aggregations** allow for powerful data analysis by grouping and summarizing search results. Aggregations provide real-time insights into search trends, query distribution, and user engagement.

❖ **Types of Aggregations for Search Analytics:**

- **Metrics Aggregations** – Calculate stats like count, average, min/max, and sum.
- **Bucket Aggregations** – Group search results into categories (e.g., by date or keyword).
- **Pipeline Aggregations** – Perform calculations on aggregated data, such as moving averages.
- **Significant Terms Aggregation** – Identifies terms that frequently appear in search queries.
- **Geo-Aggregations** – Analyze searches based on geographical locations.

Search Analytics

❖ **Visualizing Search Analytics in Kibana**

Kibana, the visualization tool for Elasticsearch, enables users to create dashboards and reports to **monitor search analytics** effectively. These insights help refine search performance and user experience.

❖ **Key Kibana Features for Search Analytics:**

- **Search Query Dashboards** – Track top queries, trends, and user engagement.
- **Heatmaps & Graphs** – Visualize search performance, response times, and error rates.
- **Real-Time Monitoring** – Detect slow queries and optimize performance.
- **User Interaction Analysis** – Identify high-traffic searches and improve ranking strategies.
- **Custom Alerts & Reports** – Automate alerts for unusual search patterns or failures.

theknowledgeacademy

Module 5:
Data Management

- ✓ Data Management
- ✓ Down Sampling
- ✓ Snapshot and Restore

Data Management

❖ **Introduction to Data Management in Elasticsearch**

Effective **data management** in Elasticsearch ensures optimized storage, performance, and retrieval of indexed data. Managing large-scale datasets efficiently is crucial for maintaining search speed and reducing resource consumption.

❖ **Key Aspects of Data Management:**

- **Efficient Indexing** – Structuring data for fast queries and minimal storage overhead.
- **Data Retention Policies** – Managing the lifecycle of data to balance performance and cost.
- **Shard Allocation & Optimization** – Distributing data across nodes for scalability.
- **Storage Optimization** – Using compression and tiered storage to reduce space usage.
- **Backup & Recovery** – Ensuring data safety through snapshots and restore mechanisms.

Data Management

❖ **Index Management in Elasticsearch**

Indexes in Elasticsearch store and organize documents for fast retrieval. Managing indexes properly improves search performance and reduces operational overhead.

❖ **Key Components of Index Management:**

- **Index Lifecycle Management (ILM)** – Automates index rollovers, retention, and deletion.
- **Index Templates** – Predefine settings and mappings for consistent indexing.
- **Index Aliases** – Create virtual indexes for seamless query routing.
- **Force Merging & Shrinking** – Optimize older indexes by reducing segment count.
- **Read-Only & Frozen Indexes** – Minimize resource usage for infrequent searches.

Data Management

❖ **Data Retention Strategies**

Managing **data retention** is essential to balance performance, cost, and storage efficiency. Elasticsearch provides various strategies to handle historical data effectively.

❖ **Common Data Retention Strategies:**

- **Time-Based Retention** – Automatically delete or archive old indices.
- **Tiered Storage Architecture** – Move less frequently accessed data to lower-cost storage.
- **Snapshot & Restore** – Back up data before deletion for long-term storage.
- **Index Rollovers** – Continuously create new indices to prevent performance degradation.
- **Retention Policy Automation** – Use ILM to define data expiration and transition rules.

Data Management

❖ **Optimizing Storage and Performance**

As data grows, **storage optimization** becomes crucial to maintain Elasticsearch's speed and efficiency. Various techniques help reduce disk usage while ensuring fast search operations.

❖ **Techniques for Storage Optimization:**

- **Compression** – Use built-in compression algorithms to reduce index size.
- **Sparse Fields** – Minimize unnecessary fields to reduce storage overhead.
- **Frozen Indices** – Store rarely used data in low-resource modes.
- **Segment Merging** – Reduce the number of segments in an index for faster searches.
- **Dedicated Hot-Warm-Cold Architecture** – Optimize storage by categorizing data based on access frequency.

Down Sampling

❖ **Introduction to Down Sampling in Elasticsearch**

Down sampling is a technique used to reduce the volume of stored data while preserving essential information. It helps optimize storage and query performance by summarizing large datasets.

❖ **Why Down Sampling is Important:**

- **Reduces Storage Costs** – Stores only critical data points.
- **Improves Query Performance** – Queries run faster on smaller datasets.
- **Optimizes Long-Term Storage** – Keeps historical trends without excess data.
- **Enhances Resource Efficiency** – Frees up system resources for active data.
- **Useful for Time-Series Data** – Ideal for logs, metrics, and monitoring systems.

Down Sampling

❖ **How Down Sampling Works**

Down sampling aggregates detailed data into summarized records, reducing the overall index size while maintaining useful insights.

❖ **Common Down Sampling Techniques:**

- **Averaging** – Stores the average value over a specific time period.
- **Min/Max Aggregation** – Keeps track of the highest and lowest values.
- **Bucket Sampling** – Groups data into buckets for statistical analysis.
- **Percentiles & Medians** – Retains key distribution points for analytics.
- **Custom Aggregations** – Uses scripted aggregations to retain necessary data points

Down Sampling

❖ **Implementing Down Sampling in Elasticsearch**

Down sampling can be implemented using **rollup jobs**, which store summarized data while preserving query capabilities.

❖ **Steps to Implement Down Sampling:**

Step 1: Define a **rollup job** using Elasticsearch APIs.

Step 2: Select indices, time intervals, and aggregation methods.

Step 3: Configure the storage location for the rolled-up data.

Step 4: Execute the rollup job and monitor its progress.

Step 5: Query the down-sampled data using **rollup search** to maintain insights.

theknowledgeacademy

Down Sampling

❖ **How it works**

Step 1: Create a Date Field Index:

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X PUT "https://localhost:9200/sales_data" -H "Content-Type: application/json" -d '{
  "mappings": {
    "properties": {
      "sale_date": {
        "type": "date"
      },
      "product": {
        "type": "text"
      },
      "amount": {
        "type": "float"
      }
    }
  }
}' -k
```

Down Sampling

Step 2: Insert some sample data into the index.

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X POST "https://localhost:9200/sales_data/_bulk" -H "Content-Type: application/json" -d '
{"index":{"_id":"1"}}
{"sale_date":"2022-01-01T10:00:00","product":"Laptop","amount":1000}
{"index":{"_id":"2"}}
{"sale_date":"2022-01-01T12:00:00","product":"Smartphone","amount":800}
{"index":{"_id":"3"}}
{"sale_date":"2022-01-02T09:00:00","product":"Tablet","amount":500}
{"index":{"_id":"4"}}
{"sale_date":"2022-01-02T14:00:00","product":"Headphones","amount":200}
' -k
```

Down Sampling

Step 3: Perform Down Sampling with Date Histogram Aggregation

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X GET "https://localhost:9200/sales_data/_search" -H "Content-Type: application/json" -d '{
  "size": 0,
  "aggs": {
    "sales_over_time": {
      "date_histogram": {
        "field": "sale_date",
        "interval": "day"
      },
      "aggs": {
        "total_sales": {
          "sum": {
            "field": "amount"
          }
        }
      }
    }
  }
}' -k
```

Down Sampling

Step 4: Output


```
{
   "aggregations": {
     "sales_over_time": {
       "buckets": [
         {
           "key_as_string": "2022-01-01T00:00:00.000Z",
           "key": 1640995200000,
           "doc_count": 2,
           "total_sales": {
             "value": 1800
           }
         },
         {
           "key_as_string": "2022-01-02T00:00:00.000Z",
           "key": 1641081600000,
           "doc_count": 2,
           "total_sales": {
             "value": 700
           }
         }
       ]
     }
   }
}
```

Snapshot and Restore

❖ **Introduction to Snapshot and Restore**

Snapshot and Restore in Elasticsearch provides a way to back up and recover data efficiently. Snapshots capture indices, cluster metadata, and settings, allowing seamless restoration when needed.

❖ **Importance of Snapshot and Restore:**

- **Disaster Recovery** – Prevents data loss due to hardware failures or accidental deletions.
- **Cluster Migration** – Moves data between clusters or across environments.
- **Historical Data Archiving** – Stores snapshots for long-term data retention.
- **Upgrades and Rollbacks** – Enables safe version upgrades with rollback options.
- **Optimized Storage** – Supports incremental snapshots to save space.

Snapshot and Restore

❖ **Creating Snapshots in Elasticsearch**

A **snapshot** is a backup of indices and cluster settings stored in a repository. Elasticsearch supports **incremental snapshots**, meaning only new or changed data is stored after the first backup.

❖ **Steps to Create a Snapshot:**

Step 1: Register a snapshot repository (e.g., filesystem, cloud storage).

Step 2: Configure snapshot settings, including retention policies.

Step 3: Execute the snapshot process via API or automated tools.

Step 4: Monitor the snapshot status to ensure successful completion.

Step 5: Store snapshots securely in distributed storage for redundancy.

theknowledgeacademy

Snapshot and Restore

❖ **How it works**

Step 1: Set up the Snapshot Repo

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X PUT "https://localhost:9200/_snapshot/my_backup" -H "Content-Type: application/json" -d '{
  "type": "fs",
  "settings": {
    "location": "/mnt/snapshots",
    "compress": true
  }
}' -k
```

Step 2: Create a snapshot of your index

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X PUT "https://localhost:9200/_snapshot/my_backup/snapshot_1?wait_for_completion=true" -k
```

Snapshot and Restore

Step 3: Output

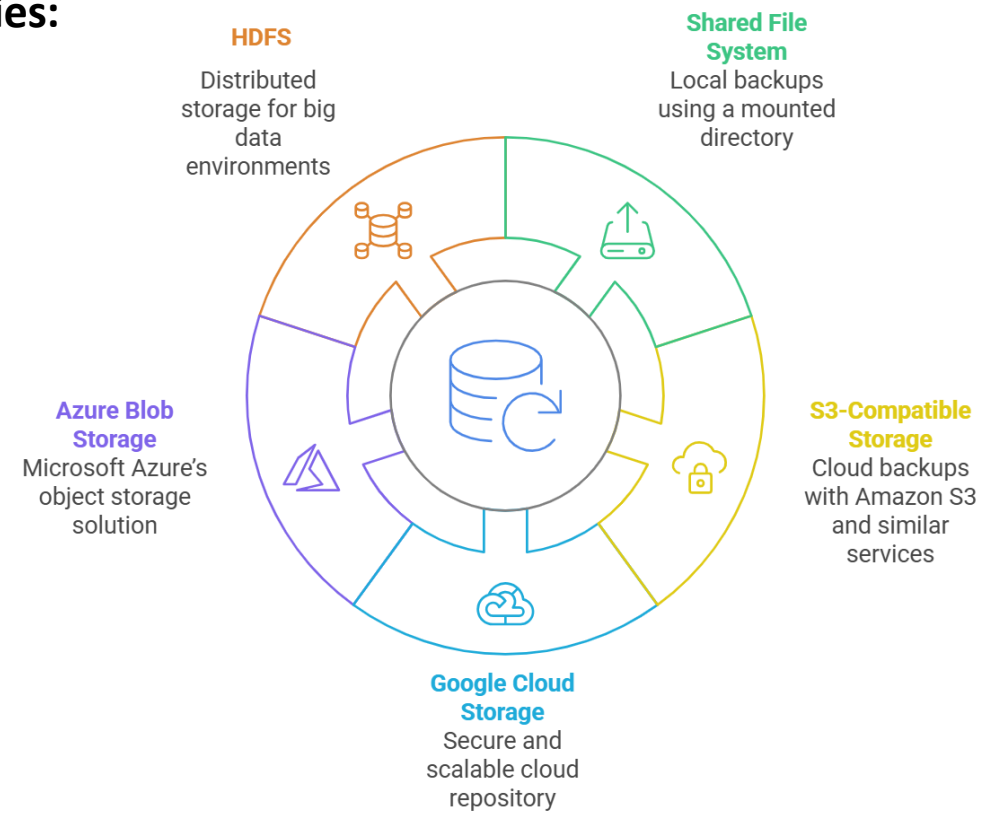

```
{
   "accepted": true,
   "snapshot": {
     "snapshot": "snapshot_1",
     "uuid": "some-unique-uuid",
     "version": "7.x.x",
     "indices": [
       "library"
     ],
     "state": "SUCCESS",
     "start_time": "2022-01-01T00:00:00",
     "end_time": "2022-01-01T00:01:00",
     "duration_in_millis": 60000
   }
}
```

Snapshot and Restore

❖ Snapshot Repositories

A **snapshot repository** is where Elasticsearch stores snapshots. Various storage types can be used to ensure backup reliability and accessibility.

❖ Types of Snapshot Repositories:



theknowledgeacademy

Snapshot and Restore

❖ **Restoring Data from Snapshots**

Restoring from snapshots allows Elasticsearch clusters to recover lost data or migrate between environments. The restore process can be **full or partial**, depending on the need.

❖ **Steps to Restore a Snapshot:**

theknowledgeacademy

Snapshot and Restore

❖ **How it works**

Step 1: Restore the library index from the snapshot:

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X POST "https://localhost:9200/_snapshot/my_backup/snapshot_1/_restore" -H "Content-Type: application/json" -d '{
  "indices": "library"
}' -k
```

Step 2: Output

```
{
   "accepted": true,
   "restore": {
     "snapshot": "snapshot_1",
     "indices": [
       "library"
     ],
     "state": "SUCCESS"
   }
}
```

Snapshot and Restore

❖ **Automating Snapshots with ILM**

Index Lifecycle Management (ILM) helps automate **snapshot creation** and **data retention**. This ensures backups are taken consistently without manual intervention.

❖ **Benefits of Automating Snapshots:**

- **Scheduled Backups** – Ensures regular snapshots without manual execution.
- **Retention Policies** – Automatically deletes old snapshots based on defined rules.
- **Disaster Recovery Readiness** – Reduces downtime in case of failures.
- **Reduced Human Error** – Eliminates risks associated with manual backup processes.
- **Efficient Resource Utilization** – Optimizes storage use with incremental snapshots.

theknowledgeacademy

Module 6:
APIs and Developer Docs

- ✓ REST APIs
- ✓ Elasticsearch Clients
- ✓ Painless Scripting Language
- ✓ Plugins and Integrations

REST APIs

❖ Introduction to Elasticsearch REST APIs

Elasticsearch provides a **RESTful API** that allows developers to interact with the system using standard HTTP methods. It enables data indexing, searching, updating, and management through simple API calls.

❖ Why Use Elasticsearch REST APIs?

- **Ease of Integration** – Works with any programming language that supports HTTP requests.
- **Scalability** – Supports distributed operations across multiple nodes.
- **Flexibility** – Allows fine-grained control over data and queries.
- **Automation** – Enables DevOps teams to manage clusters programmatically.
- **Security & Access Control** – Can be secured with authentication and role-based access.

REST APIs

❖ **Core REST API Endpoints**

Elasticsearch REST APIs follow a structured format, using HTTP methods such as **GET, POST, PUT, DELETE** for interacting with data.

❖ **Commonly Used REST API Endpoints:**

- Indexing Data (POST /index/_doc) – Adds a document to an index.
- Retrieving Data (GET /index/_doc/{id}) – Fetches a document by ID.
- Updating Data (POST /index/_update/{id}) – Modifies an existing document.
- Deleting Data (DELETE /index/_doc/{id}) – Removes a document from an index.
- Searching Data (GET /index/_search) – Executes queries to retrieve relevant documents.

REST APIs

❖ **Managing Clusters with REST APIs**

Elasticsearch REST APIs also provide powerful endpoints for **cluster and index management**, ensuring system health and performance.

❖ **Important Cluster Management APIs:**

- Cluster Health (GET /_cluster/health) – Checks the status of the cluster.
- Nodes Info (GET /_nodes) – Retrieves details about all nodes in the cluster.
- Index Settings (GET /index/_settings) – Displays or modifies index configurations.
- Index Aliases (POST /_aliases) – Manages index aliases for flexible queries.
- Snapshot & Restore (PUT /_snapshot/repository/snapshot) – Creates backups of indices and settings.

Elasticsearch Clients

❖ Introduction to Elasticsearch Clients

Elasticsearch provides **official clients** for various programming languages, allowing developers to integrate Elasticsearch seamlessly into their applications. These clients handle **low-level REST API interactions**, making development easier.

❖ Why Use Elasticsearch Clients?

- **Simplifies API Calls** – No need to manually handle HTTP requests.
- **Optimized for Performance** – Efficient connection pooling and request handling.
- **Error Handling & Retries** – Built-in mechanisms for handling failures.
- **Works with Multiple Languages** – Supports popular programming environments.
- **Secure Communication** – Supports authentication, encryption, and role-based access.

Elasticsearch Clients

❖ **Official Elasticsearch Clients**

Elasticsearch provides official clients for multiple programming languages, each optimized for performance and integration.

❖ **Common Elasticsearch Clients:**

- **Java Client** – Uses a high-level API for efficient indexing and querying.
- **Python (elasticsearch-py)** – A widely used client for Python applications.
- **JavaScript (@elastic/elasticsearch)** – Ideal for Node.js applications.
- **Go Client (go-elasticsearch)** – Optimized for performance and concurrency.
- **.NET Client (Elasticsearch.Net & NEST)** – Supports .NET applications.

Each client follows the same Elasticsearch REST API structure, ensuring consistency across languages.

theknowledgeacademy

Elasticsearch Clients

❖ **Elasticsearch on Python**

Step 1: Install Elasticsearch `pip install elasticsearch`

Step 2: Connect to the ElasticSearch instance

```
from elasticsearch import Elasticsearch

es = Elasticsearch(["https://localhost:9200"], basic_auth=("elastic", "YOUR_PASSWORD"), verify_certs=False)

if es.ping():
    print("Connected to Elasticsearch")
else:
    print("Could not connect to Elasticsearch")
```

Elasticsearch Clients

Step 3: Index a document for your selected Index

```
document = {
    "product_name": "Elasticsearch Cookbook",
    "price": 35.99,
    "release_date": "2022-05-01"
}

es.index(index="products", id=2, document=document)
```

Step 4 : Search for products with the word "Cookbook."

```
response = es.search(index="products", query={"match": {"product_name": "Cookbook"}})
print(response["hits"]["hits"])
```

Elasticsearch Clients

❖ **Choosing the Right Elasticsearch Client**

Selecting the right Elasticsearch client depends on your application's requirements, performance needs, and technology stack.

❖ **Factors to Consider:**

- **Programming Language** – Use the client that best fits your development environment.
- **Performance Needs** – For high-throughput applications, use a client with optimized request handling.
- **Ease of Use** – Choose a client with a high-level API for simpler implementation.
- **Security Features** – Ensure the client supports authentication and encryption.
- **Community & Support** – Prefer officially maintained clients for regular updates and fixes.

Painless Scripting Language

❖ Introduction to Painless Scripting

Painless is Elasticsearch's default scripting language, designed for **safe, fast, and flexible** execution of scripts within queries, aggregations, and document updates. It provides a secure way to perform calculations and transformations directly inside Elasticsearch.

❖ Why Use Painless?

- **Optimized for Performance** – Runs efficiently within Elasticsearch.
- **Secure Execution** – Sandboxed to prevent harmful operations.
- **Versatile Usage** – Works in queries, ingestions, and updates.
- **Java-Like Syntax** – Easy to learn for Java developers.
- **Dynamic Typing Support** – Allows flexible data handling.

Painless Scripting Language

❖ **How it works**

Step 1: Use a painless script to update a document. For example, increase the price of a product by 10%.

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X POST "https://localhost:9200/products/_update/1" -H "Content-Type: application/json" -d '{
  "script": {
    "source": "ctx._source.price = ctx._source.price * 1.1",
    "lang": "painless"
  }
}' -k
```

Step 2: Retrieve the updated output

```
C:\Users\training>curl -u elastic:YOUR_PASSWORD -X GET "https://localhost:9200/products/_doc/1?pretty" -k
```

Painless Scripting Language

Step 3: Output


```
{
   "_index": "products",
   "_id": "1",
   "_version": 2,
   "_seq_no": 1,
   "_primary_term": 1,
   "_source": {
     "product_name": "Elasticsearch Essentials",
     "price": 32.99,
     "release_date": "2021-01-01"
   }
}
```

Painless Scripting Language

❖ **Using Painless in Elasticsearch**

Painless scripts can be used for **custom calculations, field modifications, and conditional filtering** within Elasticsearch queries and updates.

❖ **Common Use Cases:**

- **Custom Scoring in Queries** – Modify relevance scoring dynamically.
- **Data Transformation** – Create new fields or adjust existing ones.
- **Aggregation Processing** – Perform calculations in bucket aggregations.
- **Conditional Updates** – Apply logic-based document modifications.

You can use this script to updating a document field using Painless

```
POST my_index/_update/1
{
  "script": {
    "source": "ctx._source.views += 1"
  }
}
```

Plugins and Integrations

❖ Introduction to Elasticsearch Plugins

Elasticsearch supports **plugins** to extend its core functionality, adding features such as **custom analysis, security enhancements, monitoring, and more**. These plugins help optimize and tailor Elasticsearch for specific use cases.

❖ Types of Elasticsearch Plugins:

- **Analysis Plugins** – Provide custom text analyzers and tokenizers (e.g., analysis-icu).
- **Security Plugins** – Enhance authentication, encryption, and role-based access (e.g., x-pack-security).
- **Monitoring Plugins** – Enable performance tracking and logging (e.g., metricbeat).
- **Ingest Plugins** – Process and transform data before indexing (e.g., ingest-attachment).

Plugins can be installed using:

```
bin/elasticsearch-plugin install <plugin-name>
```

Plugins and Integrations

❖ **Integrations with External Systems**

Elasticsearch integrates seamlessly with various tools and platforms to enhance data ingestion, visualization, and analytics.

❖ **Common Elasticsearch Integrations:**

- **Logstash** – Collects, processes, and ingests data into Elasticsearch.
- **Kibana** – Provides powerful dashboards and visualization tools.
- **Beats** – Lightweight data shippers for logs, metrics, and security events.
- **Apache Kafka** – Streams real-time data for indexing and analysis.
- **SQL & BI Tools** – Connects with Tableau, Power BI, and other reporting tools.

*the*knowledgeacademy

The World's Largest Global Training Provider

theknowledgeacademy.com

info@theknowledgeacademy.com

/The.Knowledge.Academy.Ltd

/TKA_Training

/the-knowledge-academy

/TheKnowledgeAcademy

Congratulations